MIT 8103 - Evidence Appendix (screenshots)

Case study: NaijaMart Online. Each image is rendered from the unedited tool output stored in the matching portfolio's evidence/ folder in the GitHub repository (source file named in each image).

Portfolio 1

Portfolio 1 - constraints reject invalid data; trigger audits stock changes

source: portfolio-1-database-design/evidence/03_constraint_tests_output.txt

```
--- 1. Duplicate e-mail (UNIQUE) ---
psql:/tmp/run_1456.sql:5: ERROR:  duplicate key value violates unique constraint "customer_email_key"
DETAIL:  Key (email)=(user1@example.com) already exists.
--- 2. Negative stock (CHECK) ---
psql:/tmp/run_1456.sql:7: ERROR:  new row for relation "product" violates check constraint "product_stock_qty_check"
DETAIL:  Failing row contains (1, SKU-00001, Classic Handbag 1, 2, 185864.75, -5, t).
--- 3. Order for a customer that does not exist (FOREIGN KEY) ---
psql:/tmp/run_1456.sql:9: ERROR:  insert or update on table "orders" violates foreign key constraint "orders_customer...
DETAIL:  Key (customer_id)=(999999) is not present in table "customer".
--- 4. Invalid order status (CHECK) ---
psql:/tmp/run_1456.sql:11: ERROR:  new row for relation "orders" violates check constraint "orders_status_check"
DETAIL:  Failing row contains (1, 19538, 2025-09-27 10:48:13.058121, LOST, 871948.70).
--- 5. Zero quantity on an order line (CHECK) ---
psql:/tmp/run_1456.sql:13: ERROR:  new row for relation "order_item" violates check constraint "order_item_quantity_c...
DETAIL:  Failing row contains (1, 2, 0, 100.00).
--- 6. Duplicate product in the same order (composite PRIMARY KEY) ---
psql:/tmp/run_1456.sql:16: ERROR:  duplicate key value violates unique constraint "order_item_pkey"
DETAIL:  Key (order_id, product_id)=(1, 45) already exists.
--- 7. Delivered before shipped (table CHECK) ---
psql:/tmp/run_1456.sql:18: ERROR:  new row for relation "delivery" violates check constraint "chk_delivery_dates"
DETAIL:  Failing row contains (15157, 3, Sendbox, TRK000000003, DELIVERED, 2025-04-20 01:32:53.006989, 2025-04-19 01:...
--- 8. Trigger test: stock change is audited automatically ---
UPDATE 1
 audit_id | product_id | old_qty | new_qty |         changed_at         | changed_by
----------+------------+---------+---------+----------------------------+------------
```

Figure: Constraints and trigger: every invalid change is rejected; the stock change is audited.

Portfolio 1 - generated data volumes

source: portfolio-1-database-design/evidence/02_seed_output.txt

```
     t      | count
------------+--------
 category   |     12
 product    |    300
 customer   |  20000
 payment    | 174379
 delivery   | 164295
 orders     | 200000
 order_item | 500000
(7 rows)
```

Figure: Data volumes generated.

Portfolio 2

Portfolio 2 - Q1 execution plan BEFORE indexing (Seq Scan)

source: portfolio-2-query-optimisation/evidence/02_explain_BEFORE_indexes.txt

```
 ##### Q1 customer history
(1 row)
                                                              QUERY PLAN
---------------------------------------------------------------------------------------------------------------------------------------
 Limit  (cost=5168.73..5169.31 rows=5 width=29) (actual time=24.368..35.744 rows=5 loops=1)
   Buffers: shared hit=3199
   ->  Gather Merge  (cost=5168.73..5169.66 rows=8 width=29) (actual time=24.366..35.740 rows=5 loops=1)
         Workers Planned: 2
         Workers Launched: 2
         Buffers: shared hit=3199
         ->  Sort  (cost=4168.71..4168.72 rows=4 width=29) (actual time=13.573..13.574 rows=2 loops=3)
               Sort Key: order_date DESC
               Sort Method: quicksort  Memory: 25kB
               Buffers: shared hit=3199
               Worker 0:  Sort Method: quicksort  Memory: 25kB
               Worker 1:  Sort Method: quicksort  Memory: 25kB
               ->  Parallel Seq Scan on orders  (cost=0.00..4168.67 rows=4 width=29) (actual time=6.502..13.422 rows=4 loops=3)
                     Filter: (customer_id = 777)
                     Rows Removed by Filter: 66663
                     Buffers: shared hit=3127
 Planning:
   Buffers: shared hit=83
 Planning Time: 0.525 ms
 Execution Time: 35.861 ms
(20 rows)
        section
```

Figure: Q1 before indexing: Parallel Seq Scan.

Portfolio 2 - Q1 execution plan AFTER indexing (Index Scan)

source: portfolio-2-query-optimisation/evidence/04_explain_AFTER_indexes.txt

```
 ##### Q1 customer history
(1 row)
                                                                 QUERY PLAN
---------------------------------------------------------------------------------------------------------------------------------------...
 Limit  (cost=0.42..22.51 rows=5 width=29) (actual time=0.040..0.084 rows=5 loops=1)
   Buffers: shared hit=8
   ->  Index Scan using idx_orders_customer_date on orders  (cost=0.42..44.59 rows=10 width=29) (actual time=0.038..0.080 rows=...
         Index Cond: (customer_id = 777)
         Buffers: shared hit=8
 Planning:
   Buffers: shared hit=143
 Planning Time: 0.863 ms
 Execution Time: 0.164 ms
(9 rows)
        section
```

Figure: Q1 after indexing: Index Scan.

Portfolio 2 - measured timings (best of 3)

source: portfolio-2-query-optimisation/evidence/timing_summary.csv

```
query,before_ms,after_ms,speedup
Q1,19.573,0.177,110.6x
Q2 (index tried),235.362,247.645,1.0x
Q2 (materialized view),235.362,0.145,1623x
Q3,15.905,2.405,6.6x
Q4,54.967,16.398,3.4x
Q5a,104.575,46.361,2.3x
```

Figure: Timing summary (best of 3).

Portfolio 3

Portfolio 3 - failed order is rolled back completely

source: portfolio-3-transactions-concurrency/evidence/01_commit_rollback.output.txt

```
 2. FAILED ORDER - customer wants 500 units but only 97 exist: the ENTIRE transaction is rolled back
(1 row)
BEGIN
INSERT 0 1
INSERT 0 1
psql:/tmp/run_1494.sql:33: ERROR:  new row for relation "product" violates check constraint "product_stock_qty_check"
DETAIL:  Failing row contains (5, SKU-00005, Lite Notebook 5, 6, 144087.16, -403, t).
ROLLBACK
      section       | new_orders_left_behind | orphan_lines | stock_p5_unchanged
--------------------+------------------------+--------------+--------------------
 AFTER FAILED ORDER |                      0 |            0 |                 97
(1 row)
                           section
```

Figure: Failed order rolled back completely.

Portfolio 3 - LOST UPDATE: two buyers, final stock 9 instead of 8

source: portfolio-3-transactions-concurrency/evidence/concurrency_demos_output.txt

```
################ DEMO 1 - LOST UPDATE: two buyers each buy 1 of 10 items. Correct final stock = 8 ################
----- Session A (lost_update_A.sql) -----
11:36:53.462 | A read stock = 10
11:36:55.492 | A committed: wrote 9
----- Session B (lost_update_B.sql) -----
11:36:53.462 | B read stock = 10
11:36:53.491 | B committed: wrote 9
----- Final state -----
 product_id | stock_qty
------------+-----------
          1 |         9
          2 |        10
```

Figure: Lost update: final stock 9 instead of 8.

Portfolio 3 - FIX with SELECT ... FOR UPDATE: B waits, final stock 8

source: portfolio-3-transactions-concurrency/evidence/concurrency_demos_output.txt

```
################ DEMO 1 FIX (b) - pessimistic locking SELECT ... FOR UPDATE. Correct final stock = 8 ################
----- Session A (fix_forupdate_A.sql) -----
11:37:24.456 | A locked row, read stock = 10
11:37:26.465 | A committed: wrote 9
----- Session B (fix_forupdate_B.sql) -----
11:37:25.060 | B asks for the row lock...
11:37:26.465 | B got the lock, read stock = 9
11:37:26.469 | B committed: wrote 8
----- Final state -----
 product_id | stock_qty
------------+-----------
          1 |         8
          2 |        10
```

Figure: Fix with SELECT ... FOR UPDATE: final stock 8.

Portfolio 3 - isolation levels: non-repeatable read vs repeatable read

source: portfolio-3-transactions-concurrency/evidence/concurrency_demos_output.txt

```
################ DEMO 2a - NON-REPEATABLE READ under READ COMMITTED (writer sets stock 10 -> 5 between the reader's t...
----- Session A (isolation_rc_reader.sql) -----
11:37:32.583 | READER (READ COMMITTED) first read: stock = 10
11:37:35.589 | READER (READ COMMITTED) second read: stock = 5
----- Session B (isolation_writer.sql) -----
11:37:33.967 | WRITER sets stock = 5 and commits
----- Final state -----
 product_id | stock_qty
------------+-----------
          1 |         5
          2 |        10
(2 rows)
################ DEMO 2b - REPEATABLE READ gives the reader a stable snapshot (second read still 10 although stock is...
----- Session A (isolation_rr_reader.sql) -----
11:37:42.316 | READER (REPEATABLE READ) first read: stock = 10
11:37:45.321 | READER (REPEATABLE READ) second read: stock = 10
----- Session B (isolation_writer.sql) -----
11:37:44.570 | WRITER sets stock = 5 and commits
----- Final state -----
 product_id | stock_qty
------------+-----------
          1 |         5
          2 |        10
```

Figure: Isolation levels.

Portfolio 3 - DEADLOCK detected and resolved

source: portfolio-3-transactions-concurrency/evidence/concurrency_demos_output.txt

```
################ DEMO 3 - DEADLOCK: A locks product 1 then wants 2; B locks 2 then wants 1. PostgreSQL aborts ONE of them ...
----- Session A (deadlock_A.sql) -----
11:38:08.263 | A locked product 1
11:38:10.266 | A now wants product 2
psql:/tmp/run_1245.sql:9: ERROR:  deadlock detected
DETAIL:  Process 1315 waits for ShareLock on transaction 1129; blocked by process 1324.
Process 1324 waits for ShareLock on transaction 1128; blocked by process 1315.
HINT:  See server log for query details.
CONTEXT:  while updating tuple (3,39) in relation "product"
11:38:11.269 | A committed
----- Session B (deadlock_B.sql) -----
11:38:08.876 | B locked product 2
11:38:10.879 | B now wants product 1
11:38:11.272 | B script finished (if the deadlock victim: its transaction was rolled back)
----- Final state -----
 product_id | stock_qty
------------+-----------
          1 |         9
          2 |         9
```

Figure: Deadlock detected by PostgreSQL.

Portfolio 4

Portfolio 4 - aggregation pipeline (revenue by state)

source: portfolio-4-nosql/evidence/03_mongo_operations_output.txt

```
=== 5. AGGREGATION PIPELINE - revenue by state (30,000-order sample) ===
{
  _id: 'Edo',
  orders: 3462,
  revenue: 2904839299.73
}
{
  _id: 'Oyo',
  orders: 3523,
  revenue: 2885413505.81
}
{
  _id: 'Kano',
  orders: 3480,
  revenue: 2870636526.81
}
{
  _id: 'Enugu',
  orders: 3449,
  revenue: 2851368536.48
```

Figure: MongoDB aggregation pipeline.

Portfolio 4 - MongoDB explain(): documents examined before vs after indexes

source: portfolio-4-nosql/evidence/03_mongo_operations_output.txt

```
=== 8. INDEXING - explain('executionStats') before and after ===
-- BEFORE indexes (collection scan)
orders by customer 777   -> docsExamined=30000, keysExamined=0, returned=2, time=105 ms
orders containing prod 45 -> docsExamined=30000, keysExamined=0, returned=300, time=159 ms
phones RAM>=8           -> docsExamined=300, keysExamined=0, returned=18, time=13 ms
-- AFTER indexes
orders by customer 777   -> docsExamined=2, keysExamined=2, returned=2, time=35 ms
orders containing prod 45 -> docsExamined=300, keysExamined=300, returned=300, time=15 ms
```

Figure: explain(): documents examined before/after indexes.

Portfolio 5

```
Portfolio 5 - streaming replication, read-only replica, synchronous mode

source: portfolio-5-distributed-cloud/evidence/02_replication_demo_output.txt


=================== A. Replication status as seen from the PRIMARY (pg_stat_replication) ===================
 application_name | client_addr |   state   | sync_state | bytes_behind
------------------+-------------+-----------+------------+--------------
 walreceiver      | 172.18.0.4  | streaming | async      |            0
(1 row)
=================== B. Roles: primary is read-write, replica is in recovery (read-only) ===================
    node    | is_replica
------------+------------
 nm-primary | f
(1 row)
    node    | is_replica
------------+------------
 nm-replica | t
(1 row)
=================== D. Writes on the replica are rejected (protects consistency: only ONE node accepts writes) ===================
ERROR:  cannot execute INSERT in a read-only transaction
=================== G. Switch to SYNCHRONOUS replication: the commit waits until the replica confirms (stronger consistency) ==...
 application_name |   state   | sync_state
------------------+-----------+------------
 walreceiver      | streaming | sync
(1 row)
   id   |            created
--------+-------------------------------
 200002 | 2026-10-24 12:01:19.590905+00
(1 row)
INSERT 0 1
=================== H. Availability vs consistency (CAP): stop the replica, then try to commit a write with a 5-second limit ==...
--> replica stopped; INSERT on primary (synchronous mode):
--> NO RESPONSE after 5 s: the primary refuses to acknowledge the commit without its synchronous standby (consistency kept, ava...
--> go back to ASYNCHRONOUS mode: the primary stays available
   id   |                note
--------+-----------------------------------
 200004 | async commit while replica is DOWN
(1 row)
```

Figure: Replication, read-only replica, synchronous mode.

Portfolio 5 - sharding across two servers, partition pruning, cross-shard query

source: portfolio-5-distributed-cloud/evidence/03_sharding_output.txt

```
=================== 2. Load 100,000 orders through the coordinator: each row is routed to its shard automat...
INSERT 0 100000
     shard     | orders
---------------+--------
 orders_shard0 |  50019
 orders_shard1 |  49981
(2 rows)
=================== 3. Proof the data is PHYSICALLY on the second server (query nm-shard2 directly) =======...
  server   | orders_stored_here
-----------+--------------------
 nm-shard2 |              49981
(1 row)
   server   | orders_stored_here
------------+--------------------
 nm-primary |              50019
(1 row)
=================== 4. Single-shard query: WHERE customer_id = 777 touches ONE shard only (partition prunin...
                  QUERY PLAN
---------------------------------------------
 Foreign Scan on orders_shard1 orders_sharded
(1 row)
                QUERY PLAN
----------------------------------------
 Seq Scan on orders_shard0 orders_sharded
   Filter: (customer_id = 778)
(2 rows)
=================== 5. Cross-shard query: aggregate over BOTH shards - result must equal the unsharded tabl...
                        QUERY PLAN
-----------------------------------------------------------
 HashAggregate
   Group Key: orders_sharded.status
   ->  Append
         ->  Seq Scan on orders_shard0 orders_sharded_1
         ->  Foreign Scan on orders_shard1 orders_sharded_2
(5 rows)
        source        | orders |   revenue
----------------------+--------+-------------
 sharded  (2 servers) | 100000 | 82174643071
 original (1 server)  | 100000 | 82174643071
```

Figure: Sharding across two servers.

Portfolio 5 - primary crash, standby promoted, no committed data lost

source: portfolio-5-distributed-cloud/evidence/04_failover_output.txt

```
=================== 3. FAILOVER: promote the standby to primary ===================
 pg_promote
------------
 t
(1 row)
    node     | is_standby | timeline
-------------+------------+----------
 nm-replica2 | f          |        2
(1 row)
=================== 4. The new primary accepts writes, and nothing committed before the crash was lost ===================
   id   |          note
--------+------------------------
 200005 | written AFTER failover
(1 row)
INSERT 0 1
 orders_still_there | replication_test_rows
--------------------+-----------------------
             200006 |                200005
(1 row)
```

Figure: Failover to a promoted standby.